

Great.  Now add **Step 3D — open_record()**.

This is the most security-sensitive part because the assignment requires:

verify HMAC before decrypting

and:

do not release plaintext after any error.

Move to the bottom of `secure_record.py` and paste:

None

```
def open_record(
    record,
    expected_version,
    expected_direction,
    expected_sequence,
    message_type,
    session_id,
    k_enc,
    k_mac,
):
    if len(record) < HEADER_SIZE + IV_SIZE + TAG_SIZE:
        raise ValueError("Record is too short")

    if len(session_id) != SESSION_ID_SIZE:
        raise ValueError("session_id must be 8 bytes")

    if len(k_enc) != AES_KEY_SIZE:
        raise ValueError("AES key must be 32 bytes")

    if len(k_mac) != 32:
        raise ValueError("MAC key must be 32 bytes")

    # Parse the fixed-size header.
    header = record[:HEADER_SIZE]

    version = header[0]
```

```

direction = header[1]
sequence = struct.unpack(
    ">Q",
    header[2:10],
)[0]
received_message_type = header[10]
ciphertext_length = struct.unpack(
    ">I",
    header[11:15],
)[0]

# Check the fields before processing the record.
if version != expected_version:
    raise ValueError("Wrong version")

if direction != expected_direction:
    raise ValueError("Wrong direction")

if sequence != expected_sequence:
    raise ValueError("Unexpected sequence number")

if received_message_type != message_type:
    raise ValueError("Wrong message type")

expected_record_length = (
    HEADER_SIZE
    + IV_SIZE
    + ciphertext_length
    + TAG_SIZE
)

if len(record) != expected_record_length:
    raise ValueError("Invalid record length")

# Extract IV, ciphertext, and tag.
iv_start = HEADER_SIZE

```

```

iv_end = iv_start + IV_SIZE

ciphertext_start = iv_end
ciphertext_end = ciphertext_start + ciphertext_length

iv = record[iv_start:iv_end]
ciphertext = record[ciphertext_start:ciphertext_end]
received_tag = record[ciphertext_end:]

# Verify that the IV matches the expected session ID and
sequence.
expected_iv = (
    session_id
    + expected_sequence.to_bytes(SEQUENCE_SIZE, "big")
)

if iv != expected_iv:
    raise ValueError("Invalid IV")

# Verify HMAC BEFORE decrypting.
mac = hmac.HMAC(k_mac, hashes.SHA256())
mac.update(header + iv + ciphertext)

try:
    mac.verify(received_tag)
except Exception:
    raise ValueError("Invalid MAC")

# Only decrypt AFTER successful MAC verification.
cipher = Cipher(
    algorithms.AES(k_enc),
    modes.CTR(iv),
)

decryptor = cipher.decryptor()
plaintext = (

```

```
        decryptor.update(ciphertext)
        + decryptor.finalize()
    )

    return plaintext
```

Why this function is important

The order is deliberately:

None

1. Parse header
2. Check version
3. Check direction
4. Check sequence number
5. Check message type
6. Check lengths
7. Check IV
8. Verify HMAC
9. THEN decrypt
10. Return plaintext

The important security rule is:

None

```
HMAC verification
    ↓
succeeds
    ↓
decrypt
    ↓
return plaintext
```

If the HMAC fails:

None

HMAC verification

↓

fails

↓

ERROR

↓

NO PLAINTEXT

That directly satisfies the requirement **"verify HMAC before decrypting."**

Sequence-number protection

This check:

None

```
if sequence != expected_sequence:  
    raise ValueError("Unexpected sequence number")
```

means the receiver expects exactly the next sequence number.

For example, if the receiver expects:

None

```
sequence = 1
```

and receives:

None

```
sequence = 0
```

that's a replay and gets rejected.

If it receives:

None

```
sequence = 2
```

when it expected 1, that's also rejected.

Direction protection

This:

```
None
if direction != expected_direction:
    raise ValueError("Wrong direction")
```

prevents a gateway→node record from being accepted as a node→gateway record.

Paste this section underneath your `seal()` function.

Don't run it yet. Once you've pasted it, tell me and we'll add **Step 3E — a test program that demonstrates normal encryption/decryption plus rejection of tampering, replay, and wrong-direction records.**

I pasted this section

Great. ☒ Now we need to add the **testing code** so we can prove Task 3 works and produce the output you'll use in your report.

Step 3E — Add the test code

Move to the **bottom of `secure_record.py`** and paste:

```
None
def run_tests():
    print("=== Task 3 Encrypt-Then-MAC Record Layer ===")

    # Use the same type of keys produced by Task 2.
    session_id = bytes.fromhex("86003dbbbc7a756e")

    k_enc = bytes.fromhex(

"eadbec73c1c4922f39531dd977d90089998f7c19bf94c17458424c7c611d56c4"
    )
```

```
k_mac = bytes.fromhex(
    "6df62a473dc59b926316e6eaf7c7784050091129b580d467b50020c9edba927c"
)

plaintext = b'{"action":"READ","path":"notes.txt"}'

# Sequence numbers begin at zero.
sequence = 0

print("\n1. Normal record")

record = seal(
    VERSION,
    DIRECTION_GATEWAY_TO_NODE,
    sequence,
    1,
    plaintext,
    session_id,
    k_enc,
    k_mac,
)

recovered = open_record(
    record,
    VERSION,
    DIRECTION_GATEWAY_TO_NODE,
    sequence,
    1,
    session_id,
    k_enc,
    k_mac,
)

print("Plaintext recovered:", recovered.decode())
```

```

print("Normal record: SUCCESS")

print("\n2. Modified header")

modified_header = bytearray(record)

# Change the message type.
modified_header[10] ^= 1

try:
    open_record(
        bytes(modified_header),
        VERSION,
        DIRECTION_GATEWAY_TO_NODE,
        sequence,
        1,
        session_id,
        k_enc,
        k_mac,
    )
    print("Modified header: ACCEPTED")
except ValueError:
    print("Modified header: REJECTED")

print("\n3. Modified ciphertext")

modified_ciphertext = bytearray(record)

# Header = 15 bytes, IV = 16 bytes.
# Modify the first ciphertext byte.
modified_ciphertext[HEADER_SIZE + IV_SIZE] ^= 1

try:
    open_record(
        bytes(modified_ciphertext),
        VERSION,

```



```

        DIRECTION_GATEWAY_TO_NODE,
        sequence,
        1,
        session_id,
        k_enc,
        k_mac,
    )
    print("Modified ciphertext: ACCEPTED")
except ValueError:
    print("Modified ciphertext: REJECTED")

print("\n4. Replay")

try:
    open_record(
        record,
        VERSION,
        DIRECTION_GATEWAY_TO_NODE,
        1,
        1,
        session_id,
        k_enc,
        k_mac,
    )
    print("Replay: ACCEPTED")
except ValueError:
    print("Replay: REJECTED")

print("\n5. Wrong direction")

try:
    open_record(
        record,
        VERSION,
        DIRECTION_NODE_TO_GATEWAY,
        sequence,

```

```

        1,
        session_id,
        k_enc,
        k_mac,
    )
    print("Wrong direction: ACCEPTED")
except ValueError:
    print("Wrong direction: REJECTED")

print("\n6. Invalid MAC")

modified_tag = bytearray(record)
modified_tag[-1] ^= 1

try:
    open_record(
        bytes(modified_tag),
        VERSION,
        DIRECTION_GATEWAY_TO_NODE,
        sequence,
        1,
        session_id,
        k_enc,
        k_mac,
    )
    print("Invalid MAC: ACCEPTED")
except ValueError:
    print("Invalid MAC: REJECTED")

if __name__ == "__main__":
    run_tests()

```

What this tests

You will get six important demonstrations:

None

Normal record
Modified header
Modified ciphertext
Replay
Wrong direction
Invalid MAC

The expected result is:

None

Normal record: SUCCESS
Modified header: REJECTED
Modified ciphertext: REJECTED
Replay: REJECTED
Wrong direction: REJECTED
Invalid MAC: REJECTED

One important detail

We're using the **gateway** → **node** keys from your successful Task 2 run:

None

session_id
K_g2n_enc
K_g2n_mac

That's why the values in the test code correspond to your Task 2 output.

The record layer is using:

None

K_g2n_enc → AES-256-CTR
K_g2n_mac → HMAC-SHA-256

which demonstrates the requirement for **separate encryption and MAC keys**.

Save it

After pasting:

Ctrl + O → Enter → Ctrl + X

Then run:

```
None  
python secure_record.py
```

Paste the entire output here. We'll check it against every Task 3 requirement before moving on.

Yes.  **That is exactly what you want for Task 4.**

You have:

```
None  
collected 6 items
```

and every required test shows:

```
None  
PASSED
```

Specifically:

1.  Valid handshake + bidirectional messages
2.  Modified ciphertext rejected
3.  Modified authenticated header rejected
4.  Replay rejected
5.  Reflected record rejected
6.  Reflected handshake rejected

And the important final result is:

```
None  
6 passed
```

Your output is cut off at:

None

```
=====
```

so **before submitting**, I recommend running it once more and capturing the **bottom of the output**, where pytest should display something like:

None

```
6 passed in X.XXs
```

For your report

Take a screenshot of the complete:

None

```
PYTHONPATH=. pytest -v tests
```

output.

You can state that the automated Task 4 suite contains **six separate tests**, with each test asserting either successful operation for the valid case or rejection of the specific adversarial modification.

This satisfies the Task 4 requirement that **each test assert a specific safe failure**.

do I need to add pycahe to the github repo

No. **You should not add `__pycache__` to your GitHub repository.**

`__pycache__` contains Python-generated bytecode files created when you run your `.py` files. They are not part of your assignment source code.

You should also **not commit `.venv/`**.

Add these to `.gitignore`

From your `hw2` directory:

None

```
vim .gitignore
```

Add:

None

```
__pycache__/  
*.pyc  
.venv/  
.pytest_cache/
```

Then save with:

None

```
Esc  
:wq  
Enter
```

Your repository should contain things like:

None

```
baseline_ctr.py  
handshake.py  
secure_record.py  
ffdhe3072.pem  
tests/  
    test_modified_ciphertext.py  
    test_modified_header.py  
    test_reflected_handshake.py  
    test_reflected_record.py  
    test_replay.py  
    test_valid_handshake.py  
.gitignore
```

and **not**:

None

```
__pycache__/  
.venv/  
.pytest_cache/
```

One thing to check: **ffdhe3072.pem** should be included if your assignment expects the generated group file to be loaded from the repository.

give me a read me format on how to run tasks 0-4

Absolutely. Here's a clean README you can put directly into your repository. It covers **Lab Preparation / Task 0 through Task 4**, including the virtual environment, required packages, DH parameter file, and how to run the tests.

README.md

CSCE 465/765 Homework — Agent Security

Environment

This project was developed and tested on:

- Ubuntu 24.04
- Python 3.12.3
- OpenSSL 3.0+
- **cryptography==49.0.0**
- **pytest==9.1.1**

Project Structure

None

```
hw2/  
├─ baseline_ctr.py  
├─ handshake.py  
├─ secure_record.py  
├─ ffdhe3072.pem
```

```
├── tests/
│   ├── test_modified_ciphertext.py
│   ├── test_modified_header.py
│   ├── test_reflected_handshake.py
│   ├── test_reflected_record.py
│   ├── test_replay.py
│   └── test_valid_handshake.py
└── .gitignore
```

Lab Preparation / Task 0

1. Navigate to the project

```
None
cd ~/Downloads/HW1/HW1/csce465-agentsec
```

2. Create the Python virtual environment

```
None
python3 -m venv .venv
```

Activate it:

```
None
source .venv/bin/activate
```

If Ubuntu reports that `ensurepip` is unavailable, the `python3.12-venv` package must be installed by an administrator.

3. Install required packages

With the virtual environment active:

None

```
python -m pip install --upgrade pip  
python -m pip install cryptography==49.0.0 pytest==9.1.1
```

4. Navigate to the homework directory

None

```
cd ~/Downloads/HW1/HW1/csce465-agentsec/hw2
```

5. Generate the Diffie–Hellman parameter file

The parameter file is required for Task 2.

None

```
openssl genpkey -genparam -algorithm DH -pkeyopt group:ffdhe3072  
-out ffdhe3072.pem
```

Verify it:

None

```
openssl dhparam -in ffdhe3072.pem -text -noout | head -3
```

The output should indicate:

None

```
DH Parameters: (3072 bit)  
GROUP: ffdhe3072
```

6. Check the environment

Before running the tasks:

None

```
python3 --version
```

```
openssl version  
pip show cryptography pytest
```

Task 1 — AES-CTR Without Authentication

Task 1 demonstrates that encryption by itself does not provide integrity or replay protection.

Run:

```
None  
python baseline_ctr.py
```

The program demonstrates:

- The original command.
- An equal-length modified command.
- The XOR difference between the plaintexts.
- The resulting ciphertexts.
- Successful decryption of the modified ciphertext.
- Replay of the same ciphertext.

A successful run should show the receiver processing the modified command and processing the original ciphertext twice during the replay demonstration.

Task 2 — Authenticated Diffie–Hellman Handshake

Task 2 implements the authenticated finite-field Diffie–Hellman handshake using:

- `ffdhe3072`
- 3072-bit RSA signing keys
- RSA-PSS with SHA-256
- Fresh DH ephemeral values
- Fresh 16-byte nonces

- Length-prefixed transcript encoding
- SHA-256 transcript hashing
- HMAC-SHA-256 key derivation
- Separate keys for each communication direction

Run:

None

```
python handshake.py
```

A successful run should show:

None

```
Gateway signature: VALID
Node signature: VALID
Shared secret matches: True
Derived keys match: True
Handshake: SUCCESS
```

The rejection tests should also show **REJECTED** for invalid signatures, changed nonces, changed public values, unexpected identities, malformed transcripts, bad declared lengths, and reflected handshake messages.

Task 3 — Encrypt-Then-MAC Record Layer

Task 3 implements the secure record layer using:

- AES-256-CTR
- HMAC-SHA-256
- Separate keys for each direction
- Sequence numbers
- Session ID and sequence number for the IV
- Encrypt-then-MAC authentication
- Constant-time MAC verification

Run:

None

```
python secure_record.py
```

The test program checks:

1. Normal Gateway → Node record
2. Sequence number progression
3. Replay protection
4. Modified header
5. Modified ciphertext
6. Invalid MAC
7. Wrong direction
8. Wrong message type
9. Unexpected sequence number
10. Malformed record
11. Node → Gateway separate direction/key operation

A successful run ends with:

None

```
=== Task 3 Test Result ===  
All tests passed: True  
Task 3 test suite: SUCCESS
```

Task 4 — Adversarial Automated Tests

Task 4 contains separate automated pytest files in the `tests/` directory.

The tests cover:

1. Valid handshake and bidirectional messages
2. Modified ciphertext
3. Modified authenticated header
4. Replayed record
5. Reflected record
6. Reflected/invalid handshake signature

Because the tests import files from the parent `hw2` directory, run them with:

None

```
PYTHONPATH=. pytest -v tests
```

A successful test run should show:

None

```
collected 6 items
```

```
tests/test_modified_ciphertext.py ... PASSED
tests/test_modified_header.py ... PASSED
tests/test_reflected_handshake.py ... PASSED
tests/test_reflected_record.py ... PASSED
tests/test_replay.py ... PASSED
tests/test_valid_handshake.py ... PASSED
```

```
6 passed
```

Each adversarial test uses assertions or `pytest.raises()` to verify that the attack results in a safe rejection.

Running Everything

After activating the virtual environment:

None

```
cd ~/Downloads/HW1/HW1/csce465-agentsec
source .venv/bin/activate
cd hw2
```

Run Task 1:

None

```
python baseline_ctr.py
```

Run Task 2:

None

```
python handshake.py
```

Run Task 3:

None

```
python secure_record.py
```

Run Task 4:

None

```
PYTHONPATH=. pytest -v tests
```

Notes

Do not commit generated Python cache files or the virtual environment. The following should be ignored by Git:

None

```
__pycache__/  
*.pyc  
.venv/  
.pytest_cache/
```

The `ffdhe3072.pem` file is required by the Task 2 implementation and should be included in the repository if required by the assignment submission instructions.

This is ready to save as `README.md` in your `hw2` repository.